

■# TODO

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

Product And Architecture

1. Consider implementing system logic on the client side as well as the server side, so the platform can hint what to bid during training.
2. Add ACBL alert-rule support. The system should eventually auto-analyze whether a call is ACBL-alertable. Since ACBL rules change over time, this needs versioned/current rules data. For the near term, each gadget rule can include an explicit `alertable` field in its public meaning.
3. Add an LLM-assisted gadget builder. Users should be able to describe a gadget in natural bridge language, have the platform call an LLM to draft the structured gadget file, then inspect, validate, test, edit, and approve it before use.
4. Treat the product as layered modules. Layer 1 is custom convention/system editing, file sharing, system/gadget execution, system-note reports, and LLM conversion between notes and structured files. Layer 2 is the tournament/inter-user platform built on top of Layer 1.
5. Add human-readable system-note report generation from structured system/gadget files.
6. Add LLM-driven conversion from human-readable system notes to structured system and gadget files.
7. Add a hand/deal workspace for manual hand entry, random dealing, PBN import, auction stepping, and bidding against a selected system.

Implementation Backlog

1. Build the web user interface.
2. Add account and saved-system support.
3. Add system sharing, import, export, copying, and forking.
4. Add full bridge auction legality checking.
5. Add double/redouble legality and auction completion detection.
6. Add client-side training hints.
7. Add automated ACBL alert analysis.
8. Add LLM-assisted gadget drafting from natural-language descriptions.
9. Add custom evaluator plugin interface.
10. Add partner obligations.
11. Add competitive double library.
12. Add PBN import support and deal validation.
13. Design a systematic gadget-file naming scheme. Each gadget should live in its own directory, and the platform should eventually assign or validate unique gadget IDs so imported gadgets do not conflict. Consider including the gadget ID in filenames where useful.
14. Build the next operable milestone: basic 2/1 plus uncontested after-1NT gadgets. This should include a practical 1NT response and continuation library, not only the current heart-transfer sample.

15. Add structured competition among after-1NT routes such as Jacoby transfers, Texas transfers, Smolen, invitational signoffs, and game-forcing auctions. Selection must evaluate hand, auction, environment, and semantic facts through approved algorithms or plugin hooks.
16. Add central derivation of bridge-relative context: our side, their side, opener, responder, overcaller, advancer, seat number, favorable/unfavorable vulnerability, balancing seat, and related equivalences.
17. Replace the current flexible `SemanticFact` trace with an ontology-backed `SemanticState` based on `docs/semantic\_ontology.md`.
18. Add semantic conflict and ambiguity diagnostics for cases such as multiple meanings of `4N`, incompatible shown ranges, unresolved obligations, and overlapping gadget rules.
19. Implement formal transfer, relay, forcing, agreed-suit/trump, notrump-focus, competitive-interference, and slam/keycard state. This should support cases such as doubled-transfer retransfer sequences and RKCB versus quantitative `4N` without enumerating every possible auction path.
20. Define and implement the Bridge System Language roadmap. BSL should be a formal authoring language that compiles to validated IR/YAML; it should not replace direct IR/YAML editing for advanced users.
21. Add a compiler and validator pipeline: BSL source to normalized IR, IR schema validation, source-location-aware diagnostics, and executable engine loading.
22. Add GUI authoring surfaces for form editing, BSL editing, and IR/YAML inspection. The GUI should not assume BSL is always easier than IR; it should support the right surface for the task.

Design Notes To Revisit

1. Public partnership meaning and internal training origin are separate output layers.
2. A selected bid should report which rule and gadget produced it.
3. Judgmental choices should report compared candidate rules, not only the final selected rule.
4. Gadgets and rules need author metadata.
5. Human-authored IDs should stay short, with fully qualified origin names generated by the engine.
6. Selection reasoning must be structured and machine-readable. Natural-language display text can be generated from structured criterion results, but it should not be the source of truth.
7. Custom gadget/rule selection algorithms need a practical plugin interface.
8. LLM-generated gadgets must remain drafts until validated and approved. The structured gadget file is the source of truth, not the natural-language prompt.
9. Keep tournament/inter-user features separated from the system execution engine so the engine remains useful as a standalone authoring, training, and analysis tool.
10. A basic system, such as 2/1, is a user-facing concept. Technically it is just a gadget. Do not add unnecessary backend fields or object types that distinguish "basic system" from other gadgets unless a concrete implementation requirement appears.
11. Rule IDs should name what the rule does, not generic concepts like `meaning`. A rule is a formal call rule with optional selection, meaning, semantic effects, and default-policy behavior.
12. Before using any default behavior for an undefined auction, the engine should search all active gadgets globally for matching rules.
13. The ontology is a system-level contract. Gadget YAML, engine code, generated system notes, alert analysis, and LLM-assisted gadget drafting should share the same formal vocabulary.
14. The engine executes validated IR. BSL, natural-language notes, LLM drafts, and GUI forms are authoring layers that must compile or validate before use.